

MCT 506

Computer Software Engineering II

Object oriented software design, implementation and testing. Team software specification and management. Cross-platform tools and GUI development. Advanced software algorithms and architecture. Software engineering practice and methods.

Lecture 1: Object Oriented Software Design

Some specific topics to cover:

- a) Classes, objects, inheritance, polymorphism
- b) Encapsulation, abstraction
- c) Design patterns
- d) System modelling and UML

Classes and Objects:

(<https://www.freecodecamp.org/news/object-oriented-concepts/>)

<https://www.freecodecamp.org/news/object-oriented-programming-in-python-interview-questions/#:~:text=Conclusion-,What%20is%20OOP?,way%20that%20could%20cause%20issues.>

<https://www.youtube.com/watch?v=SiBw7os-zI>

[https://www.freecodecamp.org/news/how-to-use-oop-in-python/#:~:text=Object%2Doriented%20programming%20\(OOP\)%20offers%20several%20benefits%20when%20organizing,color%2C%20model%2C%20and%20name.](https://www.freecodecamp.org/news/how-to-use-oop-in-python/#:~:text=Object%2Doriented%20programming%20(OOP)%20offers%20several%20benefits%20when%20organizing,color%2C%20model%2C%20and%20name.)

Class: Blueprint for a component (e.g., Motor)

Object: Instance of a class (e.g., leftMotor, rightMotor)

Inheritance: Create a base class (Sensor) and derive specific sensor classes (TemperatureSensor, UltrasonicSensor)

Example: TemperatureSensor inherits properties and methods from Sensor

Polymorphism: Methods with same name, different behavior (e.g., readData() in TemperatureSensor and UltrasonicSensor)

Enables generic programming (e.g., Sensor::readData() works for any sensor type)

Encapsulation: Hide internal details, expose necessary info (e.g., Motor class hides speed control logic, exposes setSpeed() method)

Protects data, improves modularity

Abstraction: Focus on essential features (e.g., Sensor class abstracts different sensor types, provides common interface)

Simplifies system design, improves code reusability

Example code in c++

```
class Sensor {  
  
public:  
  
    virtual float readData() = 0; // abstract method  
  
};
```

```
class TemperatureSensor : public Sensor {  
  
public:  
  
    float readData() { /* read temperature */ }  
  
};
```

```
class UltrasonicSensor : public Sensor {  
  
public:  
  
    float readData() { /* read distance */ }  
  
};
```

Let's consider a robotic arm example:

Classes and Objects:

- `Joint` class (represents a joint in the arm)
- `joint1`, `joint2`, `joint3` objects (specific joints)

Inheritance:

- `Sensor` base class
- `PositionSensor`, `ForceSensor` derived classes (inherit `readData()` method)

Polymorphism:

- `Sensor\* sensors[] = {new PositionSensor(), new ForceSensor()};`
- `sensors[i]->readData()` calls correct method for each sensor type

Encapsulation:

- `Motor` class hides speed control logic, exposes `setSpeed(float speed)` method
- `void Motor::setSpeed(float speed) { /\* control logic \*/ }`

Abstraction:

- `Actuator` abstract class provides common interface for motors, grippers, etc.
- `class Motor : public Actuator { /\* implementation \*/ };`

Another example: Autonomous Vehicle

- `Vehicle` class with `drive()`, `stop()` methods

- 'AutonomousCar', 'AutonomousDrone' classes inherit and implement specific behavior

C++ example:

```
class Actuator {
```

```
public:
```

```
    virtual void activate() = 0;
```

```
};
```

```
class Motor : public Actuator {
```

```
public:
```

```
    void activate() { /* activate motor */ }
```

```
};
```

```
class Gripper : public Actuator {
```

```
public:
```

```
    void activate() { /* activate gripper */ }
```

```
};
```

```
void controlActuator(Actuator* actuator) {
```

```
    actuator->activate(); // polymorphic call
```

```
}
```

Implementation in python

```
# Classes and Objects
```

```
class Joint:
```

```
def __init__(self, name):
```

```
    self.name = name
```

```
    self.angle = 0
```

```
def set_angle(self, angle):
```

```
    self.angle = angle
```

```
# Inheritance
```

```
class Sensor:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def read_data(self):
```

```
        pass # abstract method
```

```
class PositionSensor(Sensor):
```

```
    def __init__(self, name):
```

```
        super().__init__(name)
```

```
    def read_data(self):
```

```
        return f'{self.name} position: 45 degrees'
```

```
class ForceSensor(Sensor):
```

```
    def __init__(self, name):
```

```

        super().__init__(name)

    def read_data(self):

        return f"{self.name} force: 10 N"

# Polymorphism

sensors = [PositionSensor("Joint 1"), ForceSensor("Gripper")]

for sensor in sensors:

    print(sensor.read_data())

# Encapsulation

class Motor:

    def __init__(self, name):

        self.__speed = 0

        self.name = name

    def set_speed(self, speed):

        if 0 <= speed <= 100:

            self.__speed = speed

        else:

            print("Invalid speed")

    def get_speed(self):

```

```
    return self.__speed
```

```
motor = Motor("Joint 1 Motor")
```

```
motor.set_speed(50)
```

```
print(motor.get_speed()) # Output: 50
```

```
# Abstraction
```

```
from abc import ABC, abstractmethod
```

```
class Actuator(ABC):
```

```
    @abstractmethod
```

```
    def activate(self):
```

```
        pass
```

```
class Motor(Actuator):
```

```
    def activate(self):
```

```
        print("Motor activated")
```

```
class Gripper(Actuator):
```

```
    def activate(self):
```

```
        print("Gripper activated")
```

```
def control_actuator(actuator: Actuator):
```

```
    actuator.activate()
```

```
motor = Motor()
```

```
gripper = Gripper()
```

```
control_actuator(motor) # Output: Motor activated
```

```
control_actuator(gripper) # Output: Gripper activated
```

More Example on: Classes and Objects

```
class Car:
```

```
    def __init__(self, brand, model, year):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
        self.year = year
```

```
        self.mileage = 0
```

```
    def drive(self, miles):
```

```
        self.mileage += miles
```

```
    def describe_car(self):
```

```
        print(f"This car is a {self.year} {self.brand} {self.model} with {self.mileage} miles.")
```

```
my_car = Car('Toyota', 'Camry', 2015)
```

```
my_car.describe_car() # Output: This car is a 2015 Toyota Camry with 0 miles.
```

```
my_car.drive(100)
```

`my_car.describe_car()` # Output: This car is a 2015 Toyota Camry with 100 miles.

- We define a `Car` class with attributes (`brand`, `model`, `year`, `mileage`) and methods (`drive`, `describe\_car`).
- The `\_\_init\_\_` method initializes the object's attributes.
- We create a `my\_car` object and call its methods.

Inheritance Example:

```
class Vehicle:
```

```
    def __init__(self, brand, model, year):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
        self.year = year
```

```
    def describe_vehicle(self):
```

```
        print(f"This vehicle is a {self.year} {self.brand} {self.model}.")
```

```
class Car(Vehicle):
```

```
    def __init__(self, brand, model, year, doors):
```

```
        super().__init__(brand, model, year)
```

```
        self.doors = doors
```

```
    def describe_car(self):
```

```
        self.describe_vehicle()
```

```
        print(f"It has {self.doors} doors.")
```

```
my_car = Car('Toyota', 'Camry', 2015, 4)
```

```
my_car.describe_car()
```

```
# Output:
```

```
# This vehicle is a 2015 Toyota Camry.
```

```
# It has 4 doors.
```

- We define a `Vehicle` class and a `Car` class that inherits from `Vehicle`.

- The `Car` class adds a `doors` attribute and a `describe\_car` method.

- We use `super()` to call the parent class's `\_\_init\_\_` method.

Polymorphism Example:

```
class Shape:
```

```
    def area(self):
```

```
        pass
```

```
class Circle(Shape):
```

```
    def __init__(self, radius):
```

```
        self.radius = radius
```

```
    def area(self):
```

```
        return 3.14 * self.radius ** 2
```

```
class Rectangle(Shape):
```

```
    def __init__(self, width, height):
```

```
        self.width = width
```

```
self.height = height
```

```
def area(self):
```

```
    return self.width * self.height
```

```
shapes = [Circle(5), Rectangle(4, 6)]
```

```
for shape in shapes:
```

```
    print(shape.area())
```

```
# Output:
```

```
# 78.5
```

```
# 24
```

- We define a `Shape` class with an `area` method and two child classes `Circle` and `Rectangle`.
- Each child class overrides the `area` method.
- We create a list of shapes and call the `area` method on each shape.

Encapsulation Example:

```
class BankAccount:
```

```
    def __init__(self, balance=0):
```

```
        self.__balance = balance
```

```
    def deposit(self, amount):
```

```
        if amount > 0:
```

```
            self.__balance += amount
```

```

    else:

        print("Invalid deposit amount")

def get_balance(self):

    return self.__balance

account = BankAccount(1000)

account.deposit(500)

print(account.get_balance()) # Output: 1500

# Trying to access private attribute will raise an AttributeError

# print(account.__balance)

- We define a `BankAccount` class with a private `__balance` attribute.

- The `deposit` method modifies the balance, and the `get_balance` method returns it.

- We can't access the private attribute directly from outside the class.

```

Abstraction Example:

```

from abc import ABC, abstractmethod

```

```

class Payment(ABC):

    @abstractmethod

    def process_payment(self, amount):

        pass

```

```

class CreditCard(Payment):

```

```
def process_payment(self, amount):  
    print(f'Charging ${amount} to credit card')
```

```
class PayPal(Payment):  
    def process_payment(self, amount):  
        print(f'Transferring ${amount} via PayPal')
```

```
def make_payment(payment_method, amount):  
    payment_method.process_payment(amount)
```

```
credit_card = CreditCard()
```

```
paypal = PayPal()
```

```
make_payment(credit_card, 100) # Output: Charging $100 to credit card
```

```
make_payment(paypal, 200) # Output: Transferring $200 via PayPal
```

- We define an abstract `Payment` class with an abstract `process\_payment` method.
- `CreditCard` and `PayPal` classes implement the `process\_payment` method.
- The `make\_payment` function works with any payment method.

Let's create examples that demonstrate classes, objects, inheritance, polymorphism, encapsulation, and abstraction in the context of robotics.

Lets create a robotic arm with sensors and actuators.

We will create examples that demonstrate classes, objects, inheritance, polymorphism, encapsulation, and abstraction in the context of robotics.

We'll create a robotic arm with sensors and actuators.

```
# robotics.py
```

```
# Abstraction
```

```
from abc import ABC, abstractmethod
```

```
class Sensor(ABC):
```

```
    @abstractmethod
```

```
    def read_data(self):
```

```
        pass
```

```
class Actuator(ABC):
```

```
    @abstractmethod
```

```
    def activate(self):
```

```
        pass
```

```
# Classes and Objects
```

```
class Joint:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        self.angle = 0
```

```
        self.motor = Motor(name + " Motor")
```

```
    def set_angle(self, angle):
```

```
        self.angle = angle
```

```
self.motor.activate(angle)
```

```
class Motor(Actuator):
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def activate(self, angle):
```

```
        print(f"{self.name} moving to {angle} degrees")
```

```
# Inheritance and Polymorphism
```

```
class PositionSensor(Sensor):
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def read_data(self):
```

```
        return f"{self.name} position: 45 degrees"
```

```
class ForceSensor(Sensor):
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
    def read_data(self):
```

```
        return f"{self.name} force: 10 N"
```

Encapsulation

```
class RoboticArm:
```

```
    def __init__(self):
```

```
        self.joints = [Joint("Joint 1"), Joint("Joint 2")]
```

```
        self.sensors = [PositionSensor("Joint 1 Sensor"), ForceSensor("Gripper Sensor")]
```

```
    def move_joint(self, joint_index, angle):
```

```
        if 0 <= joint_index < len(self.__joints):
```

```
            self.__joints[joint_index].set_angle(angle)
```

```
        else:
```

```
            print("Invalid joint index")
```

```
    def read_sensor_data(self, sensor_index):
```

```
        if 0 <= sensor_index < len(self.__sensors):
```

```
            return self.__sensors[sensor_index].read_data()
```

```
        else:
```

```
            return "Invalid sensor index"
```

Usage

```
arm = RoboticArm()
```

```
arm.move_joint(0, 90)
```

```
print(arm.read_sensor_data(0)) # Output: Joint 1 Sensor position: 45 degrees
```

```
print(arm.read_sensor_data(1)) # Output: Gripper Sensor force: 10 N
```

This example demonstrates:

Classes and Objects: `Joint`, `Motor`, `Sensor`, `PositionSensor`, `ForceSensor`, and `RoboticArm` are classes. `arm` is an object of `RoboticArm`.

Inheritance: `PositionSensor` and `ForceSensor` inherit from `Sensor`. `Motor` implements `Actuator`.

Polymorphism: `PositionSensor` and `ForceSensor` override the `read\_data` method. `Motor` implements the `activate` method.

Encapsulation: `RoboticArm` encapsulates joints and sensors, providing methods to interact with them.

Abstraction: `Sensor` and `Actuator` are abstract classes providing a common interface for sensors and actuators.

Below is another detailed code example that demonstrates classes, objects, inheritance, polymorphism, encapsulation, and abstraction in the context of robotics.

```
# robotics.py

# Abstraction

from abc import ABC, abstractmethod

# Abstract class for sensors

class Sensor(ABC):

    @abstractmethod

    def read_data(self):

        """Read data from the sensor"""

        pass
```

```
# Abstract class for actuators

class Actuator(ABC):

    @abstractmethod

    def activate(self):

        """Activate the actuator"""

        pass
```

```
# Classes and Objects
```

```
class Joint:

    def __init__(self, name):

        self.name = name

        self.angle = 0

        self.motor = Motor(name + " Motor")

    def set_angle(self, angle):

        self.angle = angle

        self.motor.activate(angle)

    def __str__(self):

        return f"Joint {self.name} at {self.angle} degrees"

class Motor(Actuator):

    def __init__(self, name):

        self.name = name
```

```
def activate(self, angle):  
    print(f'{self.name} moving to {angle} degrees')
```

Inheritance and Polymorphism

```
class PositionSensor(Sensor):  
    def __init__(self, name):  
        self.name = name  
  
    def read_data(self):  
        return f'{self.name} position: 45 degrees'
```

```
class ForceSensor(Sensor):  
    def __init__(self, name):  
        self.name = name  
  
    def read_data(self):  
        return f'{self.name} force: 10 N'
```

Encapsulation

```
class RoboticArm:  
    def __init__(self):  
        self.__joints = [Joint("Joint 1"), Joint("Joint 2")]  
        self.__sensors = [PositionSensor("Joint 1 Sensor"), ForceSensor("Gripper Sensor")]
```

```

def move_joint(self, joint_index, angle):

    if 0 <= joint_index < len(self.__joints):

        self.__joints[joint_index].set_angle(angle)

    else:

        print("Invalid joint index")

```

```

def read_sensor_data(self, sensor_index):

    if 0 <= sensor_index < len(self.__sensors):

        return self.__sensors[sensor_index].read_data()

    else:

        return "Invalid sensor index"

```

```

def __str__(self):

    joints_str = "\n".join(str(joint) for joint in self.__joints)

    return f"Robotic Arm:\n{joints_str}"

```

Usage

```

if __name__ == "__main__":

    arm = RoboticArm()

    print(arm)

    arm.move_joint(0, 90)

    print(arm.read_sensor_data(0)) # Output: Joint 1 Sensor position: 45 degrees

    print(arm.read_sensor_data(1)) # Output: Gripper Sensor force: 10 N

```

Breakdown of the concepts:

Classes and Objects

- `Joint`, `Motor`, `Sensor`, `PositionSensor`, `ForceSensor`, and `RoboticArm` are classes.
- `arm` is an object of `RoboticArm`.
- `joint1` and `joint2` are objects of `Joint` created inside `RoboticArm`.

Inheritance

- `PositionSensor` and `ForceSensor` inherit from `Sensor`.
- `Motor` implements `Actuator`.

Polymorphism

- `PositionSensor` and `ForceSensor` override the `read\_data` method.
- `Motor` implements the `activate` method.

Encapsulation

- `RoboticArm` encapsulates joints and sensors, providing methods to interact with them.
- The `\_\_joints` and `\_\_sensors` attributes are private, and access is provided through methods.

Abstraction

- `Sensor` and `Actuator` are abstract classes providing a common interface for sensors and actuators.

Other Examples

Example 1: Bank Account System

```
class BankAccount:
```

```
    def __init__(self, account_number, balance=0):
```

```
        self.__account_number = account_number
```

```
        self.__balance = balance
```

```
    def deposit(self, amount):
```

```
        if amount > 0:
```

```
            self.__balance += amount
```

```
            print(f'Deposited ${amount}. New balance: ${self.__balance}')
```

```
        else:
```

```
            print("Invalid deposit amount")
```

```
    def withdraw(self, amount):
```

```
        if 0 < amount <= self.__balance:
```

```
            self.__balance -= amount
```

```
            print(f'Withdrew ${amount}. New balance: ${self.__balance}')
```

```
        else:
```

```
            print("Invalid withdrawal amount")
```

```
    def get_balance(self):
```

```
        return self.__balance
```

```

class SavingsAccount(BankAccount):

    def __init__(self, account_number, balance=0, interest_rate=0.02):

        super().__init__(account_number, balance)

        self.__interest_rate = interest_rate


    def add_interest(self):

        interest = self.get_balance() * self.__interest_rate

        self.deposit(interest)


account = SavingsAccount("123456789", 1000)

account.deposit(500)

account.withdraw(200)

account.add_interest()

print(f'Final balance: ${account.get_balance()}')

```

Classes and Objects:

We have two classes, `BankAccount` and `SavingsAccount`. `BankAccount` is a base class, and `SavingsAccount` is a derived class that inherits from `BankAccount`. The `account` object is an instance of `SavingsAccount`.

Encapsulation:

The `\_\_balance` attribute is private, and we use methods (`deposit`, `withdraw`, `get\_balance`) to interact with it. This encapsulates the data and provides a controlled interface to access and modify it.

Inheritance: `SavingsAccount` inherits the attributes and methods of `BankAccount` using inheritance. It also adds a new attribute `\_\_interest\_rate` and a new method `add\_interest`.

Polymorphism:

The `deposit` method is used in both `BankAccount` and `SavingsAccount` classes. In `SavingsAccount`, the `add\_interest` method uses the `deposit` method from `BankAccount`.

Abstraction:

The `BankAccount` class provides an abstract interface for bank accounts, hiding the implementation details. The `SavingsAccount` class adds specific functionality for savings accounts.

Example 2: University Management System

class Person:

```
def __init__(self, name, age):  
    self.name = name  
    self.age = age
```

class Student(Person):

```
def __init__(self, name, age, student_id):  
    super().__init__(name, age)  
    self.student_id = student_id  
    self.courses = []
```

```
def enroll_course(self, course):  
    self.courses.append(course)
```

class Instructor(Person):

```
def __init__(self, name, age, instructor_id):  
    super().__init__(name, age)
```

```
self.instructor_id = instructor_id
```

```
self.courses = []
```

```
def assign_course(self, course):
```

```
    self.courses.append(course)
```

```
class Course:
```

```
    def __init__(self, course_id, name):
```

```
        self.course_id = course_id
```

```
        self.name = name
```

```
student = Student("Ade Bala", 20, "123")
```

```
instructor = Instructor("Bola Chibuzor", 35, "I123")
```

```
course = Course("MCT511", "Software Engineering")
```

```
student.enroll_course(course)
```

```
instructor.assign_course(course)
```

```
print(f"Student: {student.name}, Courses: {[course.name for course in student.courses]}")
```

```
print(f"Instructor: {instructor.name}, Courses: {[course.name for course in instructor.courses]}")
```

Classes and Objects:

We have three classes: `Person`, `Student`, and `Instructor`. `Student` and `Instructor` inherit from `Person`. We create objects `student`, `instructor`, and `course`.

Inheritance:

`Student` and `Instructor` inherit the attributes of `Person` using inheritance. They also add their own attributes and methods.

Composition:

A `Student` has a list of `Course` objects, and an `Instructor` has a list of `Course` objects. This is an example of composition, where objects of one class contain objects of another class.

Polymorphism:

The `enroll\_course` and `assign\_course` methods are specific to `Student` and `Instructor` classes, respectively.

Example 3: Vehicle Management System

class Vehicle:

```
def __init__(self, make, model, year):
```

```
    self.make = make
```

```
    self.model = model
```

```
    self.year = year
```

```
def start_engine(self):
```

```
    print("Engine started")
```

class Car(Vehicle):

```
def __init__(self, make, model, year, doors):
```

```
    super().__init__(make, model, year)
```

```
    self.doors = doors
```

```
def open_trunk(self):
```

```
    print("Trunk opened")
```

```

class Motorcycle(Vehicle):

    def __init__(self, make, model, year, engine_capacity):

        super().__init__(make, model, year)

        self.engine_capacity = engine_capacity


    def kick_start(self):

        print("Kick started")


car = Car("Toyota", "Camry", 2020, 4)

car.start_engine()

car.open_trunk()

motorcycle = Motorcycle("Honda", "CBR500R", 2022, 500)

motorcycle.start_engine()

motorcycle.kick_start()

```

Classes and Objects:

We have three classes: `Vehicle`, `Car`, and `Motorcycle`. `Car` and `Motorcycle` inherit from `Vehicle`. We create objects `car` and `motorcycle`.

Inheritance:

`Car` and `Motorcycle` inherit the attributes and methods of `Vehicle` using inheritance. They also add their own attributes and methods.

Polymorphism:

The `start\_engine` method is common to all vehicles, but `open\_trunk` and `kick\_start` are specific to `Car` and `Motorcycle`, respectively.

Question 1

Describe the difference between inheritance and polymorphism. Provide an example to illustrate your answer

Question 2

Complete the following code to create a Rectangle class that inherits from a Shape class:

```
class Shape:

    def __init__(self, color):

        self.color = color

    def area(self):

        pass
```

```
class Rectangle(Shape):

    # Complete this class
```

Question 3

Explain the concept of abstraction in object-oriented programming. Provide an example to illustrate your answer.

Question 4

Analyze the following code and identify any errors or improvements:

```
class Student:

    def __init__(self, name, age):

        self.name = name

        self.age = age

    def display_info(self):

        print(f'Name: {self.name}, Age: {self.age}')
```

```

class Course:

    def __init__(self, course_id, name):

        self.course_id = course_id

        self.name = name

    def display_info(self):

        print(f"Course ID: {self.course_id}, Name: {self.name}")

student = Student("John Doe", 20)

course = Course("CSE101", "Introduction to Programming")

student.display_info()

course.display_info()

```

Sample code to run

Let's implement the object-oriented design concepts in a Python code. This example code demonstrates classes, objects, inheritance, polymorphism, encapsulation, and abstraction:

Let's implement the object-oriented design concepts in a Python code that you can run.

Here's a detailed example code that demonstrates classes, objects, inheritance, polymorphism, encapsulation, and abstraction:

```

# Define a base class called Vehicle

class Vehicle:

    def __init__(self, brand, model, year):

        self.__brand = brand

```

```

        self.__model = model

        self.__year = year

        self.__mileage = 0

    def drive(self, miles):

        self.__mileage += miles

    def get_mileage(self):

        return self.__mileage

    def get_details(self):

        return f'{self.__year} {self.__brand} {self.__model}'

# Define a subclass called Car that inherits from Vehicle
class Car(Vehicle):

    def __init__(self, brand, model, year, doors):

        super().__init__(brand, model, year)

        self.__doors = doors

    def get_details(self):

        return super().get_details() + f' with {self.__doors} doors'

# Define a subclass called Motorcycle that inherits from Vehicle
class Motorcycle(Vehicle):

```

```

def __init__(self, brand, model, year, engine_capacity):

    super().__init__(brand, model, year)

    self.__engine_capacity = engine_capacity


def get_details(self):

    return super().get_details() + f' with {self.__engine_capacity}cc engine"

```

Define a class called Garage that demonstrates encapsulation and abstraction

```
class Garage:
```

```

    def __init__(self):

        self.__vehicles = []


    def add_vehicle(self, vehicle):

        self.__vehicles.append(vehicle)


    def display_vehicles(self):

        for vehicle in self.__vehicles:

            print(vehicle.get_details())

            print(f'Mileage: {vehicle.get_mileage()} miles")

            print()

```

Create objects and demonstrate polymorphism

```

def main():

    garage = Garage()

```

```
car = Car("Toyota", "Camry", 2020, 4)
```

```
car.drive(100)
```

```
garage.add_vehicle(car)
```

```
motorcycle = Motorcycle("Honda", "CBR500R", 2022, 500)
```

```
motorcycle.drive(50)
```

```
garage.add_vehicle(motorcycle)
```

```
garage.display_vehicles()
```

```
if __name__ == "__main__":
```

```
    main()
```

To run the code:

1. Open a text editor or an IDE (Integrated Development Environment) like PyCharm, VSCode, or Sublime Text.
2. Create a new file with a `.py` extension, for example, `oops_concepts.py`.
3. Copy and paste the code above into the file.
4. Save the file.
5. Open a terminal or command prompt and navigate to the directory where you saved the file.
6. Run the code using Python: `python oops_concepts.py`

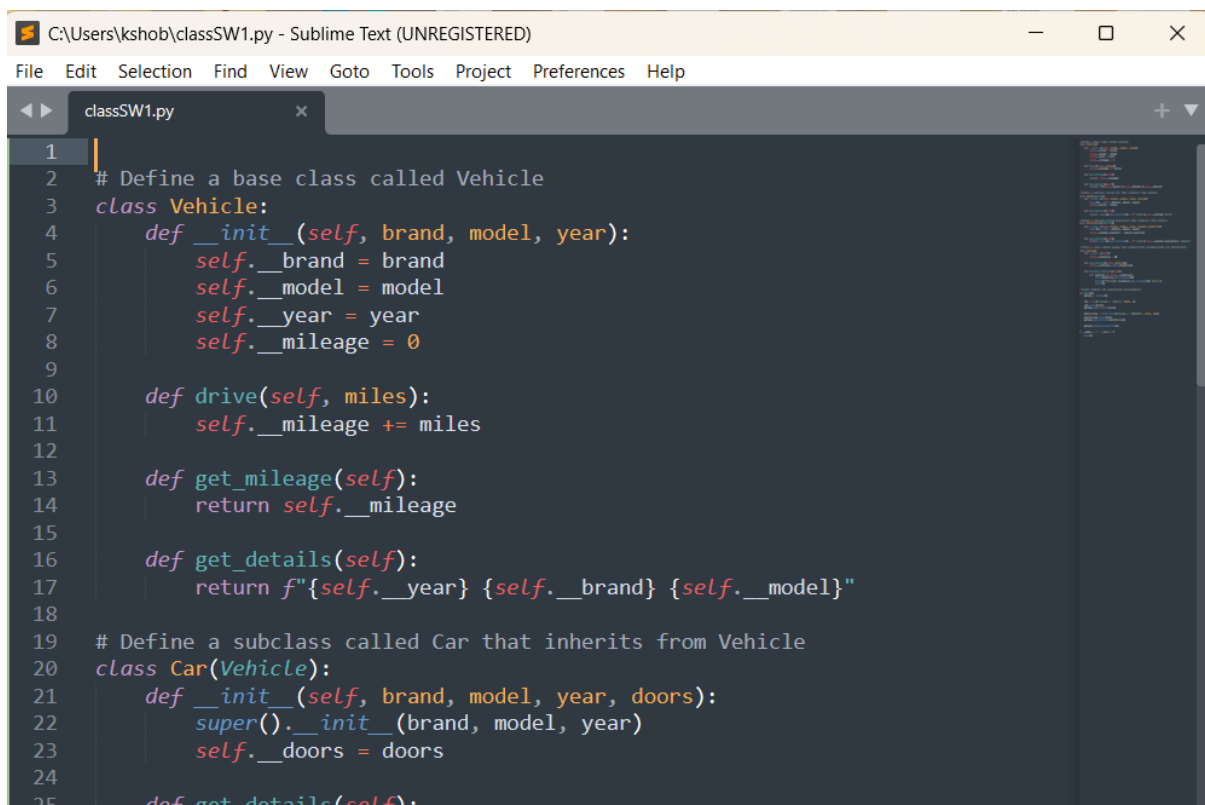
You should see the following output:

2020 Toyota Camry with 4 doors

Mileage: 100 miles

2022 Honda CBR500R with 500cc engine

Mileage: 50 miles



```
C:\Users\kshob\classSW1.py - Sublime Text (UNREGISTERED)
File Edit Selection Find View Goto Tools Project Preferences Help

classSW1.py
1
2 # Define a base class called Vehicle
3 class Vehicle:
4     def __init__(self, brand, model, year):
5         self.__brand = brand
6         self.__model = model
7         self.__year = year
8         self.__mileage = 0
9
10    def drive(self, miles):
11        self.__mileage += miles
12
13    def get_mileage(self):
14        return self.__mileage
15
16    def get_details(self):
17        return f"{self.__year} {self.__brand} {self.__model}"
18
19 # Define a subclass called Car that inherits from Vehicle
20 class Car(Vehicle):
21     def __init__(self, brand, model, year, doors):
22         super().__init__(brand, model, year)
23         self.__doors = doors
24
25     def get_details(self):
```

This code demonstrates the following object-oriented design concepts:

- Classes and objects: `Vehicle`, `Car`, `Motorcycle`, and `Garage` are classes, and `car` and `motorcycle` are objects.
- Inheritance: `Car` and `Motorcycle` inherit from `Vehicle`.
- Polymorphism: The `get\_details` method is overridden in `Car` and `Motorcycle` to provide specific details.

- Encapsulation: The 'Garage' class encapsulates a list of vehicles and provides methods to add and display vehicles.
- Abstraction: The 'Garage' class abstracts the details of vehicle management and provides a simple interface to interact with vehicles.

Output

```

Administrator: Command Prompt
Microsoft Windows [Version 10.0.26200.7922]
(c) Microsoft Corporation. All rights reserved.

C:\Windows\System32>cd..
C:\Windows>cd..
C:\>cd users
C:\Users>cd kshob
C:\Users\kshob>python claaSW1.py
python: can't open file 'C:\\Users\\kshob\\claaSW1.py': [Errno 2] No such file or directory
C:\Users\kshob>python classSW1.py
2020 Toyota Camry with 4 doors
Mileage: 100 miles
2022 Honda CBR500R with 500cc engine
Mileage: 50 miles
C:\Users\kshob>

```

Design Patterns

<https://www.geeksforgeeks.org/system-design/software-design-patterns/>

<https://www.geeksforgeeks.org/system-design/design-parking-lot-using-object-oriented-principles/>

<https://www.geeksforgeeks.org/system-design/design-data-structures-algorithms-memory-file-system/>

Design Patterns in Software Engineering

Design patterns are proven, reusable solutions to common software design problems. Think of them as *templates* you can adapt—not ready-made code, but structured ways of solving recurring issues in software engineering!

Introduction

Design patterns are reusable solutions to common problems that occur during software development. They provide a proven development paradigm to help developers create more maintainable, flexible, and scalable software systems. In this lecture, we will explore the concept of design patterns, their types, and examples.

Design patterns helps:

- Write **clean and maintainable code**
- Avoid reinventing solutions
- Improve **scalability and flexibility**
- Communicate ideas easily with other developers

What are Design Patterns?

Design patterns are general, reusable solutions to common problems that occur during software development. They are not finished designs that can be converted directly into code, but rather, they are templates or guidelines that describe how to solve a problem.

Types of Design Patterns

There are three main types of design patterns which are:

- Creational Patterns:** These patterns deal with the creation of objects and classes.
 - Singleton Pattern
 - Factory Pattern
 - Abstract Factory Pattern
 - Builder Pattern
 - Prototype Pattern
- Structural Patterns:** These patterns deal with the composition of classes and objects.
 - Adapter Pattern
 - Bridge Pattern
 - Composite Pattern
 - Decorator Pattern

- Facade Pattern
 - Flyweight Pattern
 - Proxy Pattern
- iii. **Behavioral Patterns:** These patterns deal with the interactions between classes and objects.
- Interpreter Pattern
 - Iterator Pattern
 - Mediator Pattern
 - Memento Pattern
 - Observer Pattern
 - State Pattern
 - Strategy Pattern
 - Template Method Pattern
 - Visitor Pattern

Singleton Pattern

Definition: The Singleton pattern ensures that a class has only one instance and provides a global point of access to it.

Example: A logging class that logs messages to a file.

Example: A database connection; you don't want multiple connections everywhere

Implementation:

class Singleton:

 __instance = None

 def __new__(cls): # __new__ controls object creation

 if cls.__instance is None:

 cls.__instance = super(Singleton, cls).__new__(cls) # Only one instance is created and reused

```
return cls._instance
```

Implementation:

class Singleton:

```
    _instance = None # class variable to store instance
```

```
    def __new__(cls):
```

```
        if cls._instance is None:
```

```
            print("Creating new instance...")
```

```
            cls._instance = super().__new__(cls)
```

```
        return cls._instance
```

Testing

```
obj1 = Singleton()
```

```
obj2 = Singleton()
```

```
print(obj1 is obj2) # True obj1 and obj2 point to the same object
```

Factory Pattern

Definition: The Factory pattern provides an interface for creating objects without specifying the class of object that will be created. Create objects **without specifying exact class names**

Example: A class that creates different types of database connections.

Example: Different types of vehicles

Implementation:

class DatabaseConnection:

```
    def __init__(self, type):
```

```
        self.type = type
```

```
def connect(self):
```

```
    pass
```

```
class MySQLConnection(DatabaseConnection):
```

```
    def connect(self):
```

```
        print("Connecting to MySQL database")
```

```
class PostgreSQLConnection(DatabaseConnection):
```

```
    def connect(self):
```

```
        print("Connecting to PostgreSQL database")
```

```
class DatabaseConnectionFactory:
```

```
    def create_connection(self, type):
```

```
        if type == "MySQL":
```

```
            return MySQLConnection(type)
```

```
        elif type == "PostgreSQL":
```

```
            return PostgreSQLConnection(type)
```

```
        else:
```

```
            raise ValueError("Invalid database type")
```

Implementation:

```
class Car:
```

```
    def drive(self):
```

```
        return "Driving a car"
```

```
class Bike:
```

```
    def drive(self):
```

```
        return "Riding a bike"
```

```
class VehicleFactory: # Factory decides which object to create
```

```
    @staticmethod
```

```
    def get_vehicle(vehicle_type): # User doesn't need to know class details; Improves flexibility
```

```
        if vehicle_type == "car":
```

```
            return Car()
```

```
        elif vehicle_type == "bike":
```

```
            return Bike()
```

```
        else:
```

```
            return None
```

```
# Testing
```

```
vehicle = VehicleFactory.get_vehicle("car")
```

```
print(vehicle.drive())
```

Observer Pattern (Behavioral)

When one object changes state, all dependent objects are notified.

Example: YouTube subscribers getting notified when a channel uploads.

Implementation:

```
class Subject: # Subject keeps a list of observers
```

```
    def __init__(self):
```

```

self.observers = []

def subscribe(self, observer):
    self.observers.append(observer)

def notify(self, message): # When state changes; notify all observers
    for observer in self.observers: # Loose coupling between objects
        observer.update(message)

class Observer:
    def update(self, message):
        print(f"Received update: {message}")

# Testing

subject = Subject()

obs1 = Observer()
obs2 = Observer()

subject.subscribe(obs1)
subject.subscribe(obs2)

subject.notify("New video uploaded!")

```

Strategy Pattern (Behavioral)

Define a family of algorithms and make them interchangeable.

Example:

Different payment methods (Card, PayPal)

Implementation:

```
class PaymentStrategy: # You can switch algorithms at runtime; Avoids multiple if-else conditions; Makes code flexible and extendable
```

```
    def pay(self, amount):
```

```
        pass
```

```
class CardPayment(PaymentStrategy):
```

```
    def pay(self, amount):
```

```
        return f"Paid {amount} using Card"
```

```
class PayPalPayment(PaymentStrategy):
```

```
    def pay(self, amount):
```

```
        return f"Paid {amount} using PayPal"
```

```
class ShoppingCart:
```

```
    def __init__(self, strategy):
```

```
        self.strategy = strategy
```

```
    def checkout(self, amount):
```

```
        return self.strategy.pay(amount)
```

```
# Testing
```

```
cart1 = ShoppingCart(CardPayment())
```

```
print(cart1.checkout(100))
```

```
cart2 = ShoppingCart(PayPalPayment())
```

```
print(cart2.checkout(200))
```

Decorator Pattern (Structural)

Add new functionality without modifying existing code

Example:

Adding extra features to a coffee order

Implementation:

```
def decorator(func): # Original function remains unchanged
```

```
    def wrapper():
```

```
        print("Before function call")
```

```
        func()
```

```
        print("After function call")
```

```
    return wrapper
```

```
@decorator # @decorator wraps extra behaviour around a function
```

```
def say_hello():
```

```
    print("Hello!")
```

```
# Testing
```

```
say_hello()
```

Pattern	Purpose
Singleton	One instance only
Factory	Object creation logic
Observer	Notify multiple objects
Strategy	Interchangeable behaviors
Decorator	Add functionality dynamically

View design patterns as:

Singleton: One principal in a school

Factory: A machine producing different products

Observer: WhatsApp group notifications

Strategy: Choosing transport (car, bus, bike)

Decorator: Adding toppings to pizza

Best Practices

Use design patterns to solve common problems, not to create complexity.

Keep design patterns simple and easy to understand.

Use design patterns consistently throughout the codebase.

Document design patterns and their usage.

Conclusion

Design patterns are a powerful tool for software developers to create more maintainable, flexible, and scalable software systems. By understanding and applying design patterns, developers can write better code and solve common problems more effectively.

When to Use Design Patterns

Use them when:

- You see **repeated problems**

- Code becomes hard to maintain
- You need **flexibility and scalability**

Avoid them when:

- Problem is simple (don't over-engineer)

Design Pattern Implantation in a Project

Online Food Ordering System: combination of multiple design patterns in one project

Pattern	Role in Project
Singleton	Database connection
Factory	Create different food items
Strategy	Payment methods
Observer	Notify users when order is ready

Step 1: Structure of food_ordering/

- main.py
- database.py (Singleton)
- factory.py (Factory)
- payment.py (Strategy)
- notification.py (Observer)

Step 2: Singleton Pattern (Database Connection). Only one database instance should exist. It prevents multiple unnecessary DB connections ; it saves memory and resources

class Database:

__instance = None

def __new__(cls):

```
if cls._instance is None:

    print("Connecting to database...")

    cls._instance = super().__new__(cls)

    return cls._instance
```

```
def connect(self):

    return "Database connected"
```

Step 3: Factory Pattern (Food Creation). Create food objects dynamically

```
class Pizza:
```

```
    def prepare(self):

        return "Preparing Pizza"
```

```
class Burger:
```

```
    def prepare(self):

        return "Preparing Burger"
```

```
class FoodFactory:
```

```
    @staticmethod

    def get_food(food_type): # Easy to add new food types later; Removes hardcoding

        if food_type == "pizza":

            return Pizza()

        elif food_type == "burger":

            return Burger()

        else:
```

```
return None
```

Step 4: Strategy Pattern (Payment Methods); Different payment options

```
class PaymentStrategy:
```

```
    def pay(self, amount):
```

```
        pass
```

```
class CardPayment(PaymentStrategy):
```

```
    def pay(self, amount):
```

```
        return f'Paid ₦{amount} using Card'
```

```
class CashPayment(PaymentStrategy): # Easily switch payment methods; Avoids messy if-else
```

```
    def pay(self, amount):
```

```
        return f'Paid ₦{amount} using Cash'
```

```
class PaymentContext:
```

```
    def __init__(self, strategy):
```

```
        self.strategy = strategy
```

```
    def checkout(self, amount):
```

```
        return self.strategy.pay(amount)
```

Step 5: Observer Pattern (Order Notification); Notify users when order is ready

```
class Customer:
```

```
    def update(self, message): # Multiple users get updates automatically; Loose coupling
```

```
        print(f'Customer notified: {message}')
```

```
class Order:

    def __init__(self):

        self.observers = []

    def subscribe(self, customer):

        self.observers.append(customer)

    def notify(self):

        for observer in self.observers:

            observer.update("Your order is ready!")
```

Step 6: Main Program (Integration)

```
# Singleton

db1 = Database()

db2 = Database()

print(db1.connect())

print(db1 is db2) # True


# Factory

food = FoodFactory.get_food("pizza")

print(food.prepare())


# Strategy
```

```
payment = PaymentContext(CardPayment())  
print(payment.checkout(2500))
```

```
# Observer
```

```
customer1 = Customer()
```

```
customer2 = Customer()
```

```
order = Order()
```

```
order.subscribe(customer1)
```

```
order.subscribe(customer2)
```

```
order.notify()
```

How the program works

1. **User orders food:** Factory creates Pizza or Burger
2. **System processes payment:** Strategy selects payment method
3. **Database logs order:** Singleton ensures one DB instance
4. **Order is ready:** Observer notifies all customers

Take away from this design pattern

Modularity

Each pattern handles one responsibility

Scalability

You can easily:

- Add new food types (Factory)

- Add payment methods (Strategy)
- Add notification types (Observer)

Maintainability

Changes in one part don't break others

Class work

- Add **Jollof Rice / Shawarma** to Factory
- Add **Bank Transfer payment** to Strategy
- Add **Email notification** to Observer
- Store order history in Singleton DB

System modelling and UML

Useful links

<https://www.geeksforgeeks.org/system-design/unified-modeling-language-uml-introduction/>

<https://www.visual-paradigm.com/guide/uml-unified-modeling-language/uml-practical-guide/>

System Modelling

System modelling is the process of creating a representation of a system to understand, analyse, and design its structure and behaviour. It involves creating models that capture the system's requirements, architecture, and behaviour.

Types of System Models

Structural Models: Describe the system's structure, including its components, relationships, and interfaces.

Behavioral Models: Describe the system's behavior, including its interactions, activities, and state changes.

Interaction Models: Describe the interactions between the system's components and its environment.

UML (Unified Modeling Language)

UML is a standardized language for modelling systems. It provides a set of diagrams and notations to represent a system's structure and behavior.

UML Diagrams

- **Class Diagram:** Represents the system's structure, including classes, attributes, operations, and relationships.
- **Use Case Diagram:** Represents the system's functional requirements, including actors, use cases, and relationships.
- **Sequence Diagram:** Represents the system's behavior, including interactions between objects and the order of events.
- **State Machine Diagram:** Represents the system's behavior, including states, transitions, and events.
- **Activity Diagram:** Represents the system's behavior, including activities, actions, and control flows.
- **Component Diagram:** Represents the system's structure, including components, interfaces, and relationships.
- **Deployment Diagram:** Represents the system's deployment, including nodes, components, and relationships.

UML Notations

- **Classes:** Represented by rectangles with the class name, attributes, and operations.
- **Objects:** Represented by rectangles with the object name and attributes.
- **Relationships:** Represented by lines connecting classes or objects, including associations, aggregations, and compositions.
- **Interfaces:** Represented by rectangles with the interface name and operations.
- **Use Cases:** Represented by ellipses with the use case name and description.
- **Actors:** Represented by stick figures with the actor name and description.

Benefits of UML

- **Improved Communication:** UML provides a common language for stakeholders to communicate about the system.
- **Increased Understanding:** UML helps to understand the system's structure and behavior.

- Better Design: UML enables designers to create better designs by identifying problems early.
- Reduced Errors: UML reduces errors by providing a clear and consistent representation of the system.

Best Practices for UML Use

- UML consistently: Use UML consistently throughout the project.
- Keep it simple: Keep UML diagrams simple and focused on the essential aspects of the system.
- Use meaningful names: Use meaningful names for classes, objects, and relationships.
- Use visual aids: Use visual aids, such as colors and icons, to make UML diagrams more readable.